

Vodafone Idea (VI) Telephony Integration

Complete rebuild / re-integration guide for VoicERA

Provider id: **vi** · Captured from integrated branch · 2026-09-18

Use this PDF (or the companion Markdown) as Cursor context when switching branches to implement VI from scratch. Prefer attaching the **.md** file for in-repo work.

Purpose: Complete context to re-implement the VI (Vodafone Idea / CPaaS) telephony provider from scratch on another branch. Give this document (or the companion PDF) to Cursor as context.

Provider id: vi

Display name: Vodafone Idea

Date of capture: 2026-09-18 (from voicERA tree with VI already integrated)

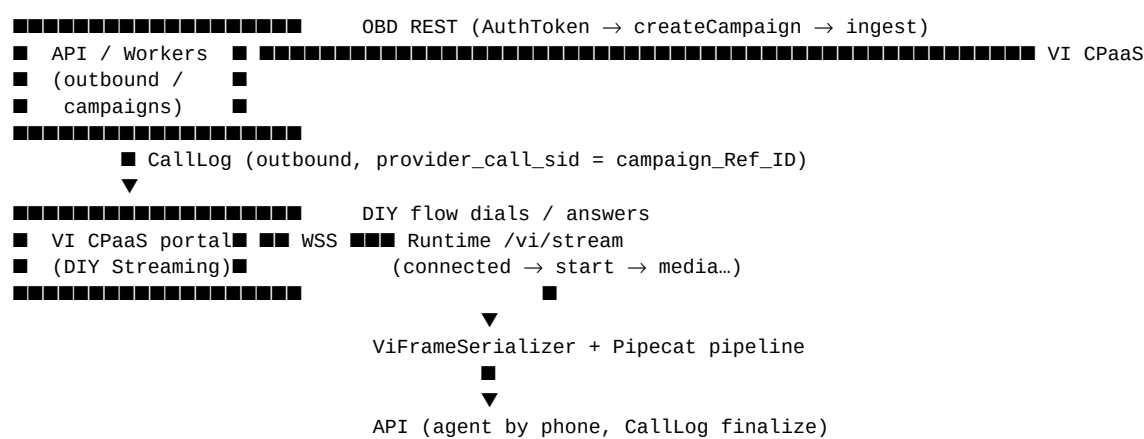
Recommendation: how to use this with Cursor

Approach	When to use
Stay in this chat + switch branch	Best if you continue the same conversation. History already contains the architecture. Ask: "Implement VI from scratch on this branch using the integration guide."
Attach this MD/PDF in a new chat	Best if you open a fresh chat, share with teammates, or expect long context compression.
Both	Recommended. Chat for iterative coding; this doc as the durable source of truth.

Prefer the .md file over PDF for Cursor: @docs/internal/vi-telephony-integration-guide.md is searchable and citable. Use the PDF for sharing outside the repo.

1. Mental model (one paragraph)

API + viobdclient place outbound/campaign calls via VI CPaaS OBD (env credentials). VI's portal DIY Streaming Object delivers answered media to runtime wss /vi/stream. Runtime resolves the agent (usually by DNI), registers a CallLog, and runs the shared Pipecat pipeline through viFrameSerializer. Registry patterns (register_telephony / register_frame_serializer) select VI like other providers, but call-control (answer XML, hangup webhooks, org ProviderAuth) is intentionally bypassed.



Critical differences vs Vobiz / Plivo

Concern	Vobiz / Plivo	VI
Answer webhook	`GET`	POST /answer` → Stream XML
Media entry	WSS /agent/{org_id}/{agent_id}	WSS /vi/stream (or legacy /vi/agent/{agent_id})
Dialing	Provider REST initiate_call	CPaaS OBD campaign API
Credentials	Org ProviderAuth	VI_OBD_USERNAME / VI_OBD_PASSWORD (+ VI_DNI, VI_FLOW_ID, ...)

Concern	Vobiz / Plivo	VI
Agent routing on answer	Query agent_id / org_id	Path / custom_parameters / DNI-CLI by-phone / VI_DEFAULT_*
Frame serializer	Vobiz / Plivo serializers	ViFrameSerializer
Recording	Pipecat AudioBuffer (+ optional provider APIs)	Pipecat AudioBuffer only
Transport	WebSocket after Stream XML	JSON-over-WebSocket PCM16 @ 8 kHz (no SIP/WebRTC)

Design invariants

1. Env credentials, not ProviderAuth — `org /auth` is unused for VI dialing.
2. No answer/hangup webhooks — media is DIY WSS only.
3. Outbound = OBD campaign — even one number is `createCampaign` + ingest.
4. Inbound and outbound media share `/vi/stream`.
5. Carrier recording unused — pipeline AudioBuffer only.
6. Application APIs are stubs so agent/number UX works without a VI application API.
7. Typo preserved: VI API field `campaignKey` (as returned by CPaaS) is used throughout.

2. File map (what to create)

Package: `apps/telephony/providers/vi/`

File	Role
<code>__init__.py</code>	Export ViClient , <code>client_or_fail</code> , ViConfig , ViOutboundQueued
<code>config.py</code>	ViAuth , ViSettings , ViConfig with <code>@register_telephony; provider="vi"</code>
<code>client.py</code>	Facade matching Vobiz/Plivo method names
<code>application.py</code>	Stub app CRUD + OBD <code>initiate_call</code> + env DNI list; VI_STUB_APP_ID = <code>"vi-env"</code>
<code>obd_client.py</code>	Real CPaaS HTTP client (auth, campaign, ingest, status)
<code>service.py</code>	<code>@register_client</code> → ViClient ; <code>@register_answer_xml("vi")</code>
<code>serializer_service.py</code>	<code>@register_frame_serializer("vi")</code> → builds ViFrameSerializer (lazy; Pipecat)
<code>serializers.py</code>	Wire protocol ↔ Pipecat frames; VI_SAMPLE_RATE = 8000
<code>xml.py</code>	Placeholder Stream XML (documents live path is <code>/vi/stream</code>)
<code>recording.py</code>	All no-ops (None)
<code>schemas.py</code>	ViOutboundQueued Pydantic model

Runtime

File	Role
<code>apps/runtime/routes/vi_telephony.py</code>	WSS <code>/vi/stream</code> , <code>/vi/agent/{id}</code> , GET <code>/vi/integration-info</code>
<code>apps/runtime/app.py</code>	<code>app.include_router(vi_telephony.router)</code>
<code>apps/runtime/services/pipecat/runners.py</code>	VI-specific sample-rate branch in <code>run_telephony_bot</code>
<code>apps/runtime/services/pipecat/telephony_rates.py</code>	<code>vi_pipeline_sample_rate()</code>

API / workers

File	Role
apps/api/app/services/outbound_call_service.py	<code>_dial_vi_outbound</code> branch when <code>provider == "vi"</code>
apps/api/app/services/campaign/campaign_call_dispatcher.py	<code>_process_vi_batch</code> , optional status poll
apps/api/app/services/agent_telephony_service.py	VI provision stub, link/unlink no-ops
apps/providers/availability.py	<code>is_authenticated("vi")</code> → env check

Shared telephony

File	Role
apps/telephony/registry.py	Discovery of <code>config</code> + <code>service</code> ; lazy <code>serializer_service</code>
apps/telephony/serializers.py	<code>create_frame_serializer("vi", ...)</code>
apps/telephony/calls.py	Generic <code>initiate_outbound</code> (VI can use; API usually bypasses)

Frontend / docs / env

File	Role
frontend/.../DeliveryStep.tsx	Delivery option <code>id: "vi"</code> (gated on authenticated)
docs/developer/guides/adding-a-telephony-provider.md	"VI exception" section
docs/developer/reference/environment-variables.md	VI env table
.env.example	VI OBD block + portal WSS notes

Tests (recommended)

- `apps/telephony/tests/test_vi_obd_client.py`
- `apps/telephony/tests/test_serializers.py` (VI cases)
- `apps/telephony/tests/test_xml.py`
- Registry/schema tests including "vi"

3. Auth

How it works

VI does **not** use org `ProviderAuth` / Integrations for dialing.

1. `ViObdClient.from_env()` reads `VI_OBD_USERNAME` / `VI_OBD_PASSWORD` (fails if missing).
2. `get_auth_token()` POSTs `{"username", "password"}` to `{obd_base}/AuthToken`.
3. Response must contain `idToken`; cache ~24h TTL, refresh 1h early (`TOKEN_TTL_SECS=86400`, `TOKEN_REFRESH_MARGIN_SECS=3600`).
4. Subsequent OBD calls send `Authorization: Bearer {idToken}`.

Gate helpers

- `vi_env_credentials_configured()` — both username and password non-empty.
- Used by: API provisioning, outbound, campaigns, `apps/providers/availability.py`.

Catalog placeholders

ViAuth / ViConfig keep `auth_id` / `auth_token` defaulting to "env" for schema symmetry with Vobiz/Plivo. They are **not** the real dial credentials.

No stream auth

There is **no** VI webhook signature / Basic auth on `/vi/stream`.

4. Environment variables

Variable	Where	Purpose
VI_OBD_USERNAME	API	OBD API username (required)
VI_OBD_PASSWORD	API	OBD API password (required)
VI_OBD_BASE_URL	API	Override base (default below)
VI_DNI	API	Fallback outbound caller ID / number inventory
VI_FLOW_ID	API	DIY flow id (else hardcoded default in <code>obd_client.py</code>)
VI_OBD_DIAL_TIMEOUT	API	<code>createCampaign</code> dialtimeout (default 30)
VI_OBD_STATUS_POLL_SECS	API	Campaign status poll interval (default 30)
VI_OBD_STATUS_POLL_MAX_ROUNDS	API	Max poll rounds (default 120 ; 0 disables)
VOICE_SERVER_BASE_URL	Runtime / API	Public host for WSS URL docs / provisioning hint
VI_DEFAULT_AGENT_ID	Runtime	Fallback agent on <code>/vi/stream</code>
VI_DEFAULT_ORG_ID	Runtime	Org for path/default loads when DNI miss
API_BASE_URL	Runtime	Runtime → API
INTERNAL_API_KEY	Runtime	by-phone + campaign notify

Default OBD base:

`https://cts.myvi.in:8443/Cpaas/api/v1/obdcampaignapi`

Default flow id constant (override with `VI_FLOW_ID`):

`68sXGL6Llic/7YdDGEtGBg==`

Ops note: Portal WSS host must not be a `.ai` domain (per `.env.example` comments). Streaming Object points at `wss://{VOICE_SERVER_BASE_URL host}/vi/stream`.

5. OBD client (`ViObdClient`) — API contract

File: `apps/telephony/providers/vi/obd_client.py`

HTTP: `httpx.client`, POST JSON, timeout 30s.

Method	Path	Purpose
<code>get_auth_token</code>	<code>AuthToken</code>	Login → <code>idToken</code>
<code>create_campaign</code>	<code>createCampaign</code>	Time-windowed OBD campaign (IST window)
<code>get_active_dni / resolve_dni</code>	<code>getActiveDNIList</code>	Caller ID; fallback VI_DNI
<code>upload_call_list / _with_fallback</code>	<code>staticCampaignDataIngestion</code>	Single MSISDN (nested / nested+top-level shapes)
<code>upload_call_list_bulk</code>	same	Chunked ingest (500/chunk)

Method	Path	Purpose
<code>get_campaign_status / _with_fallback</code>	<code>campaignstatus</code>	Poll by <code>campaign_Ref_ID</code> then <code>campaignKey</code>
<code>place_single_outbound_call</code>	<code>orchestrates</code>	Auth → DNI → create → ingest one number

Helpers

- `normalize_msisdn`: strip +, strip leading 91 if length > 10 (e.g. `+919876543210` → `9876543210`).
- `get_flow_id()`, `get_dni_from_env()`, `resolve_vi_agent_id(path, start_info)`.
- `_normalize_cpaas_base`: ensure override URLs include `/Cpaas/api/v1/`.

createCampaign *payload shape*

```
{
  "flowid": "<flow_id>",
  "fromdate": "YYYY-MM-DD",
  "todate": "YYYY-MM-DD",
  "fromtime": "HH:MM:SS",
  "totime": "HH:MM:SS",
  "dialtimeout": 30,
  "name": "voicera-YYYYMMDD-HHMMSS",
  "description": "VoicERA VI campaign",
  "retryintervaltype": 0,
  "retryintervalvalue": 5,
  "retrycount": 1
}
```

Window is IST (Asia/Kolkata). Success: HTTP 200 and `status == 1`. Extract `campaignKey` (typo) and `campaign_Ref_ID`.

Single ingest *payload shapes*

nested (preferred):

```
{"campaign_ID": "<campaignKey>", "Records": [{"dni": "...", "msisdn": "..."}]}
```

nested_and_top_level (fallback on 409):

```
{
  "campaign_ID": "<campaignKey>",
  "dni": "...",
  "msisdn": "...",
  "Records": [{"dni": "...", "msisdn": "..."}]
}
```

Success: HTTP 200 and (`data.status == "success"` OR `data.rowsAffected >= 1`).

Bulk ingest

```
{"campaign_ID": "<campaignKey>", "Records": [{"dni": "...", "msisdn": "..."}, ...]}
```

Chunk size 500. Campaign window hours for large batches: `max(1, ceil(n/50))`.

`place_single_outbound_call` *return*

Dict including at least: `campaign_Ref_ID`, `campaignKey`, `dni`, `msisdn`, `message/status` fields. API maps `campaign_Ref_ID` → `CallLog provider_call_sid / call_uuid`.

6. Pipeline / media integration

Registration

1. `load_providers()` imports each vendor `config.py` + `service.py`.
2. `load_frame_serializers()` (lazy) imports `serializer_service.py` — API must not import Pipecat serializers at startup.
3. Runtime: `create_frame_serializer("vi", stream_sid=..., call_sid=..., sample_rate=..., websocket=...)`.

Runtime endpoints (`vi_telephony.py`)

Method	Path	Handler
WebSocket	<code>/vi/stream</code>	Preferred; agent from DNI / custom params
WebSocket	<code>/vi/agent/{agent_id}</code>	Legacy path agent id
GET	<code>/vi/integration-info</code>	Portal WSS URL hints from VOICE_SERVER_BASE_URL

Mount router in `apps/runtime/app.py` alongside other telephony routers.

Session handshake

1. Accept WebSocket.
2. Read up to 20 messages / 15s until `event == "start"` (ignore `connected`).
3. Extract:
 - `call_sid` ← `start.call_id` / `call_id` / `callSid`
 - `stream_sid` ← `room_id` / `start.room_id` / `streamSid`
4. Resolve agent → create `CallLog` → `run_telephony_bot(..., provider="vi")`.
5. Force `provider="vi"` even if agent doc says otherwise.
6. Require `agent_category == "telephony"`; else close **1008**.

Agent resolution order (`_resolve_agent_full`)

1. Path `agent_id` (legacy) via `resolve_vi_agent_id`.
2. `start.custom_parameters` keys `agent_id` / `agentId` / `agent`.
3. Load with org from DNI lookup or `VI_DEFAULT_ORG_ID`.
4. Else DNI/CLI by-phone: fields `dni` / `DNI` / `cli` / `CLI` → `backend_client.get_agent_by_phone` → API `GET /agents/by-phone/{phone}`.
5. Else `VI_DEFAULT_AGENT_ID` + `VI_DEFAULT_ORG_ID`.

CallLog on media connect

Always `backend_client.create_inbound_call` (CLI=from, DNI=to, `provider_call_sid=call_sid`).

Caveat: Pre-created outbound `CallLogs` from OBD are **not** automatically linked; media session typically gets a fresh inbound `CallLog`. Custom variables from outbound `CallLog` are **not** loaded into the pipeline (unlike `/agent/...` for Vobiz/Plivo).

ViFrameSerializer *protocol*

JSON over WSS, 8 kHz PCM16, base64 payloads.

Inbound (VI → Pipecat)

Event	Effect
<code>connected</code> / <code>start</code>	Ignored (route already handled)

Event	Effect
media	base64 PCM → resample to pipeline rate → InputAudioRawFrame
dtmf	→ InputDTMFFrame
clear	→ InterruptioFrame
stop	→ EndFrame
mark	If name voicera-final , unblock exit drain

Outbound (Pipecat → VI)

Frame	Effect
AudioRawFrame	Resample to 8 kHz; chunk ≥1600 bytes, ≤51200, align 160 bytes; event: media
InterruptioFrame	event: clear
EndFrame / CancelFrame	Drain buffer → final mark (voicera-final) → wait → event: exit

Constants: **MIN_CHUNK_BYTES=1600**, **MAX_CHUNK_BYTES=51200**, **CHUNK_ALIGN_BYTES=160**, **MAX_DRAIN_SECS=30**, **DRAIN_GRACE_SECS=2**.

Sample rates (run_telephony_bot *VI branch*)

1. Wire rate = **VI_SAMPLE_RATE** (8000).
2. Build STT/TTS/LLM early to read TTS native rate.
3. **pipeline_rate** = **vi_pipeline_sample_rate(tts, wire_rate)** (8k or 16k for Silero).
4. **recording_rate** may be higher than pipeline when TTS is higher.
5. Pass serializer + rates into **run_pipeline**.

Audio path:

```

VI WSS JSON media
→ ViFrameSerializer.deserialize → InputAudioRawFrame
→ FastAPIWebsocketTransport.input()
→ Silero VAD
→ STT → LLM (+ tools/KB) → TTS
→ ViFrameSerializer.serialize → base64 PCM @ 8 kHz
→ AudioBufferProcessor → MinIO recording
→ TranscriptWriter / CallMetricsWriter → API

```

On end: finalize CallLog + **notify_campaign_call_status(..., "answered")** using the **media session's** CallLog id.

7. Outbound (single call) — OBD service

Primary path: **apps/api/app/services/outbound_call_service.py**

When **provider == "vi"**:

1. Validate telephony agent.
2. Resolve **from_number**: linked phone → **VI_DNI** → placeholder **vi-dni** (then often **+0000000000** for storage).
3. Create CallLog (**status=initiated**, then **ringing** after dial).
4. **_dial_vi_outbound** → **asyncio.to_thread(ViObdClient.from_env().place_single_outbound_call)**.
5. Persist **provider_call_sid = campaign_Ref_ID**; update **from_number** from resolved DNI if present.

Facade path: **viclient.initiate_call / application.initiate_call** also call OBD (threaded). Prefer API direct branch.

When callee answers → same **/vi/stream** media path as inbound.

8. Inbound service

No VI-specific inbound dial API. No `/answer` webhook.

1. Caller dials DNI.
2. VI DIY flow (portal Streaming Object) opens `WSS .../vi/stream`.
3. Runtime resolves agent (DNI → Numbers inventory / by-phone).
4. `POST /calls/inbound` → Pipecat pipeline.

Number inventory: local only — `list_numbers` returns `VI_DNI`; `link_number` / `unlink_number` are logged no-ops. Portal routes DIY flow by DNI.

9. Campaign service

File: `apps/api/app/services/campaign/campaign_call_dispatcher.py`

Class: `CampaignCallDispatcher`

When agent `telephony.provider == "vi"`:

1. `process_batch` claims up to `max(batch_size, 500)` queued runs (not one-by-one rate-limited dials).
2. `_process_vi_batch`:
 - Require env OBD credentials (else mark runs failed).
 - Normalize phones → MSISDNs; skip invalid.
 - `window_hours = max(1, ceil(n/50))`.
 - Thread: Auth → `resolve_dni` → `create_campaign` → `upload_call_list_bulk`.
 - Store on campaign `orchestrator_metadata`: `vi_campaign_ref_id`, `vi_campaign_key`, `vi_dni`, `vi_current_status`.
 - Create one `CallLog` per contact (`telephony_provider=vi`, shared `provider_call_sid=campaign_Ref_ID`, variables include VI campaign ids).
 - Mark queued runs processed.
3. Optional `_poll_vi_campaign_status`: poll `campaignstatus` every `VI_OBD_STATUS_POLL_SECS`, up to `VI_OBD_STATUS_POLL_MAX_ROUNDS`; update metadata dial stats; stop when queue drained or terminal.

Non-VI campaigns still call `initiate_outbound_call` per row with concurrency/rate limits.

Each answered call still hits `/vi/stream` independently.

10. Provisioning & stubs

File: `apps/api/app/services/agent_telephony_service.py`

Function	VI behavior
<code>provision_application</code>	Require env OBD creds; return <code>application_id="vi-env"</code> , <code>answer_url=wss://.../vi/stream</code> , empty hangup
<code>delete_application</code>	No-op
<code>rename_application</code>	No-op
<code>list_provider_numbers</code>	<code>ViClient.list_numbers()</code> → <code>VI_DNI</code>
<code>link_number</code> / <code>unlink_number</code>	Logged no-ops

Operator still configures DIY Streaming Object once in VI portal to `/vi/stream`.

11. Availability / frontend

- `apps/providers/availability.py`: for `provider == "vi"`, authenticated iff `VI_OBD_USERNAME` and `VI_OBD_PASSWORD` set.
 - Wizard Delivery step: option `id`: `"vi"`; disabled until catalogs show provider authenticated.
-

12. Rebuild checklist (implementation order)

Use this order when integrating from scratch:

1. **Env + docs** — add VI vars to `.env.example` and env reference docs.
 2. `obd_client.py` — `AuthToken`, `createCampaign`, `DNI`, `ingest`, `status`, `place_single`; unit tests with mocked httpx.
 3. `config.py` + `service.py` + **stubs** (`client`, `application`, `xml`, `recording`, `schemas`) — register provider in catalog.
 4. **Availability** — env-based `is_authenticated("vi")`.
 5. `serializers.py` + `serializer_service.py` — wire protocol; tests for chunking/DTMF/exit.
 6. **Runtime vi telephony** — `mount router`; `agent resolve`; `CallLog`; `run_telephony_bot`.
 7. `runners.py` / `telephony_rates.py` — VI sample-rate branch.
 8. **API outbound** — `_dial_vi_outbound` branch.
 9. **API campaigns** — `_process_vi_batch` + optional status poll.
 10. **API agent telephony** — `provision/list/link` stubs.
 11. **Frontend** — Delivery option `vi`.
 12. **Operator setup** — portal DIY Streaming Object → public WSS `/vi/stream`; attach DNI to flow; set `VI_DNI` / `VI_FLOW_ID`.
 13. **E2E verify** — inbound ring → stream; single outbound; campaign batch.
-

13. End-to-end flows (condensed)

A. Provision VI agent

1. Server has `VI_OBD_USERNAME` / `VI_OBD_PASSWORD` (and usually `VI_DNI`, `VOICE_SERVER_BASE_URL`).
2. Create telephony agent with `telephony.provider=vi`.
3. `provision_application` → stub `vi-env` + informational `wss://.../vi/stream`.
4. Attach number locally (`VI_DNI`); portal Numbers inventory routes DIY flow by DNI.
5. Operator configures DIY Streaming Object once to `/vi/stream`.

B. Single outbound

1. API `initiate_outbound_call` → `CallLog` → `place_single_outbound_call`.
2. VI dials within campaign window.
3. Answer → WSS `/vi/stream` → `agent resolve` → Pipecat.

C. Campaign outbound

1. Orchestrator/worker `process_batch` → `_process_vi_batch` (one OBD campaign, many MSISDNs).

2. CallLogs per row; optional status poll.
3. Each connected call still hits `/vi/stream` independently.

D. True inbound

1. Caller dials VI number → DIY flow → `/vi/stream`.
 2. Route by DNI → inbound CallLog → pipeline.
-

14. Known pitfalls

- **Dual CallLogs:** OBD creates outbound CallLog; media path may create another inbound CallLog. Correlation is weak (by provider sid / agent), not a shared `call_id` query param on `/vi/stream`.
 - **No hangup webhook:** missed/no-answer campaign outcomes rely on OBD status poll, not runtime hangup mapping.
 - **campaignKey typo** must be preserved when talking to VI APIs.
 - **MSISDN normalization** differs from E.164 storage in CallLogs (often `+91...` in DB, 10-digit in OBD).
 - **Do not** put VI serializer imports on the API hot path (Pipecat dependency).
 - **Do not** implement VI as a normal `/answer` Stream XML provider — that path is dead for live calls.
 - Host for WSS must be reachable by VI CPaaS (public, correct TLS, not blocked domain).
-

15. Prompt template for Cursor (new branch / new chat)

Implement Vodafone Idea (VI) telephony from scratch on this branch.

Use `@docs/internal/vi-telephony-integration-guide.md` as the full specification.

Key constraints:

- Env-based OBD credentials (not `ProviderAuth`)
- No `/answer` Stream XML for live media – WSS `/vi/stream` only
- Even single outbound = `createCampaign` + `ingest`
- Preserve `campaignKey` typo from VI APIs
- Lazy-load frame serializer (API must not import Pipecat serializers)
- Match existing Vobiz/Plivo registry patterns where possible; VI is the documented exception

Implement in this order: `obd_client` → provider package stubs/registry → serializers → runtime `vi_telephony` + sample rates → API outbound → campaigns → provision stubs → availability/frontend → tests.

16. Related existing docs in repo

- `docs/developer/guides/adding-a-telephony-provider.md` — “Vodafone Idea (VI) exception”
 - `docs/guides/concepts/telephony-model.md` — Vobiz/Plivo model (contrast)
 - `docs/developer/reference/environment-variables.md` — VI env table
 - `docs/developer/clients/telephony.md` — generic wire-level telephony
-

End of VI telephony integration rebuild guide.